



College : Shaqa University

Department: College of Computer Science and Information Technology

Name: Musallam Alomair

Student Number :444460129

Subject : artificial intelligence

Section :819 – 828

AI-Powered Pac-Man Simulation

Description (Detailed)

This project focuses on developing an autonomous Artificial Intelligence agent capable of learning to play "Pac-Man" without any pre-programmed rules or human intervention. The simulation is built using the **Python** programming language and the **Pygame** library.

The core problem addresses the challenge of navigating an unknown environment. The agent (Pac-Man) begins the game "blind," unaware of the maze layout, the location of the dots, or the behavior of the enemy. The goal is to maximize the score by collecting all dots within a strict time limit (120 seconds) while avoiding a chasing ghost.

To solve this, the project integrates two distinct computational approaches:

Reinforcement Learning (RL): Used for the player (Pac-Man) to learn decision-making through trial and error, utilizing a system of rewards and penalties.

Pathfinding Algorithms: Used for the enemy (Ghost) to create a competitive and intelligent adversary that calculates the shortest path to the player.

Problem Statement:

The primary challenge lies in enabling the agent (Pac-Man) to navigate a maze containing obstacles, goals, and a chasing enemy, given that the agent starts completely "blind" and unaware of the game rules.

Key Challenges:

Unknown Environment: The agent initially knows neither the map layout nor the locations of the dots.

Intelligent Enemy (The Ghost): A ghost chases the player using a precise algorithm to find the shortest path, making survival difficult.

Time Management: The game is constrained by a strict time limit (120 seconds), forcing the agent to find the optimal solution quickly without stalling.

Reward System: A precise reward/penalty system must be designed to guide the agent toward the goal (eating dots) and away from errors (hitting walls or colliding with the ghost).

2. Algorithms Used (Steps)

Two main algorithms govern the behavior of the simulation's entities:

Q-Learning (For Pac-Man)

This is a model-free reinforcement learning algorithm. It works through the following steps:

Initialization: A Q-Table is created, initializing all Q-values to zero. This table maps every possible state (position) to possible actions (Up, Down, Left, Right).

Observation: The agent observes the current state \$\$\$ (its position, ghost position, and nearest dot).

Action Selection: The agent chooses an action A using the **Epsilon-Greedy Strategy**:

Exploration: Choose a random action (to discover the map).

Exploitation: Choose the action with the highest Q-value (based on previous experience).

Execution & Reward: The agent performs the action, moves to a new state S' , and receives a reward R (e.g., +50 for a dot, -100 for death).

Update (Bellman Equation): The Q-value for the previous state is updated using the formula:

$$Q(S, A) = Q(S, A) + \alpha[R + \gamma \cdot \max_{A'}(Q(S', A')) - Q(S, A)]$$

B. Breadth-First Search - BFS (For the Ghost)

This algorithm ensures the ghost finds the shortest path to Pac-Man. The steps are:

Start: Add the Ghost's current position to a queue and mark it as visited.

Expansion: Dequeue the current position and check if it matches Pac-Man's position (Target).

Neighbor Inspection: If the target is not found, identify all valid neighboring cells (Up, Down, Left, Right) that are not walls.

Queuing: Add these neighbors to the queue and record the path taken to reach them.

Repetition: Repeat steps 2-4 until Pac-Man's position is reached. The ghost then takes the first step of this calculated path.

-100 points: For losing (being caught by the ghost).

-10 points: For hitting a wall.

+5 / -5 points: Heuristic rewards for moving closer to or further from the nearest dot.

Update: The table values are updated using the **Bellman Equation** to allow the agent to learn from its experiences.

Breadth-First Search (BFS) (Ghost Control)

This is a traditional **Pathfinding Algorithm** used to ensure the ghost chases the player intelligently.

Concept: The algorithm scans the grid in layers (waves) starting from the ghost's position.

Goal: To guarantee finding the **Shortest Path** available to reach Pac-Man's current location.

Implementation: Every time the ghost takes a turn, the full path to Pac-Man is calculated, and the ghost takes the first step of that optimal path, making it a difficult and precise adversary.

3. Real Example for Using Algorithm

Scenario: Pac-Man is at coordinates (1, 1) and a dot is located at (1, 2) (to his right).

Applying Q-Learning:

Current State: Pac-Man is at (1, 1). The Q-Table initially has a value of 0 for moving "Right".

Action: The agent chooses to move **Right**.

Outcome: Pac-Man arrives at (1, 2) and eats the dot.

Reward: The environment gives a positive reward of **+50**.

Update: The algorithm updates the Q-Table for the state (1, 1) and action Right.

$$\text{New Q-Value} = 0 + 0.8 * [50 + 0.9 * 0 - 0] = \mathbf{40}.$$

Result: The next time Pac-Man is at (1, 1), the Q-value for moving "Right" will be high (40), so the agent will "know" that moving Right is a good decision.

Screenshots (Steps of Running):

Figure 1: Initial State

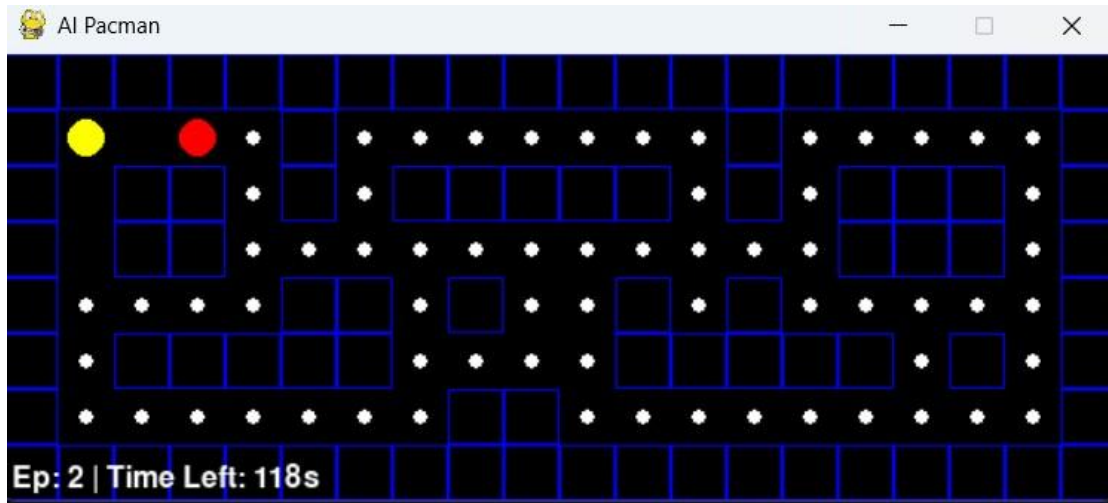


Figure 2: Learning Process

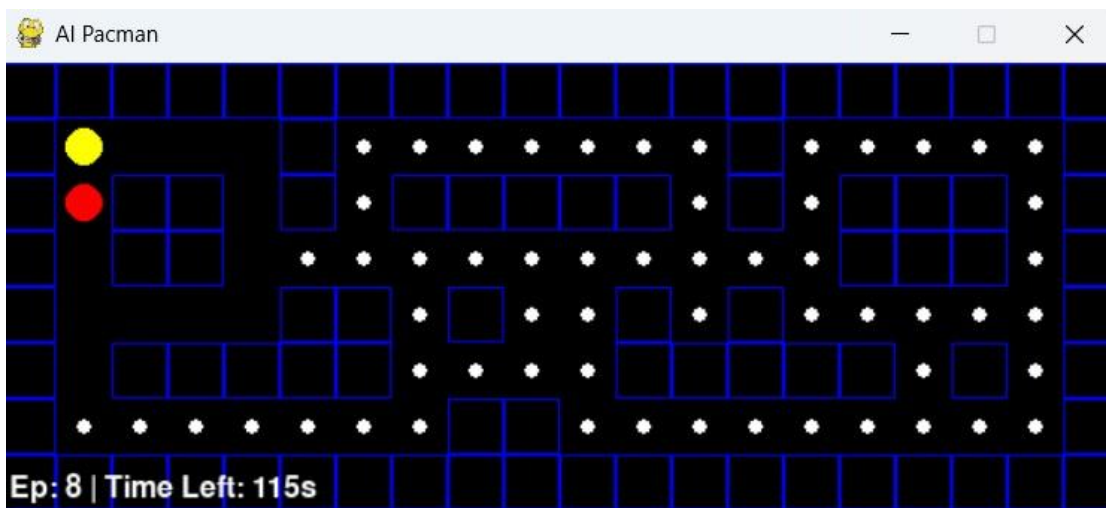


Figure 3: Winning State

```
Episode 100: WIN (Score: 710)

Optimal Path Found! Stopping Training.

=====
FINAL STATISTICS
=====
Total Episodes Run: 100
Total Optimal Paths (WINS): 1
Total Failed Attempts (LOSSES): 99
Win Rate: 1.00%
=====
```

Conclusion and Results:

By integrating the adaptive **Q-Learning** algorithm with the deterministic **BFS** algorithm, we successfully created an effective training environment.

Initially, the Pac-Man agent moves randomly and loses frequently. However, as the episodes progress and the Q-Table is updated, the agent learns to avoid walls, escape the ghost, and navigate directly toward the dots. Ultimately, the agent achieves the **Optimal Path** in record time, demonstrating the efficacy of the implemented AI model.